

Random Module in Python

Definition:

The random module is a built-in Python module used to generate random numbers and perform random operations. It helps create unpredictable values that are useful in games, simulations, password generators, lotteries, testing, and many other applications.

Why Do We Need the Random Module?

Normally, programs produce the same output for the same input. The random module allows programs to generate different results each time they run.

Importing the Module

```
import random
```

1. random.random():

The random() function returns a random floating-point number between 0.0 and 1.0.

Syntax:

```
random.random()
```

Parameters: No parameters.

Return Value: It returns a float value.

Example

```
import random
print(random.random())
```

Possible Output : 0.748521

Use Case: Generating random percentages and probabilities.

2. random.randint():

The randint() function returns a random integer within a specified range.

Syntax

```
random.randint(start, end)
```

Parameters:

Parameter	Description
start	Starting number
end	Ending number

Example:

```
import random
print(random.randint(1,10))
```

Output: 7

Use Case: Dice games and OTP generation.

3. random.choice():

The choice() function selects a random element from a sequence such as a list, tuple, or string.

Syntax

```
random.choice(sequence)
```

Example:

```
import random
fruits = [ "Apple","banana","mango"]
print(random.choice(fruits))
```

output: mango

Use Case: Random quiz questions and lucky draws.

4. random.shuffle():

The shuffle() function rearranges the elements of a list randomly.

Syntax

random.shuffle(list)

Example:

```
import random
num = [1,2,3,4,5]
random.shuffle(num)
print(num)
```

output: [3, 5, 1, 4, 2]

Important Point: Works only on mutable sequences such as lists.

5. random.sample():

Returns a specified number of unique random elements from a sequence.

Syntax

random.sample(sequence, count)

Example:

```
import random
numbers = [10, 20, 30, 40, 50]
print(random.sample(numbers, 2))
```

Output : [20, 50]

Use Case: Lottery systems and random team selection.

RegEx (Regular Expressions)

Regular Expressions (RegEx) are patterns used to search, validate, extract, and manipulate text.

Python provides the built-in re module to work with regular expressions.

Why Do We Need RegEx?

RegEx helps us:

- Validate email addresses
- Validate phone numbers
- Search patterns
- Extract data
- Replace text

Importing the Module:

```
import re
```

1. re.search():

Searches the entire string and returns the first match.

Syntax

`re.search(pattern, string)`

Example:

```
import re
text = "Python Programming"
result = re.search("Python", text)
print(result)
```

Output : <re.Match object>

Use Case: Finding keywords in documents.

2. re.match():

Checks whether the string starts with the specified pattern.

Syntax

`re.match(pattern, string)`

Example:

```
import re
print(re.match("python", "python course"))
```

output: <re.Match object>

Use Case: Input validation.

3. re.findall():

Returns all matching occurrences as a list.

Syntax

`re.findall(pattern, string)`

Example:

```
import re
text = "pyhton Django python"
print(re.findall("python", text))
```

Output: ['Python', 'Python']

Use Case: Counting occurrences of words.

4. re.sub():

Replaces matching text with another value.

Syntax

re.sub(pattern, replacement, string)

Example:

```
import re
text = "pyhton Django python"
print(re.sub("python",text))
```

Output: python Django pthon

Use Case: Data cleaning.

5. re.split():

Splits a string using a pattern.

Syntax

re.split(pattern, string)

Example

```
import re
text = "pyhton Django python"
print(re.split(", ",text))
```

output: ['Python', 'Django', 'Flask']

Common RegEx Symbols:

Symbol	Meaning
.	Any character
^	Starts with
\$	Ends with
*	Zero or more
+	One or more
?	Optional
\d	Digit
\D	Non-digit
\w	Word character
\W	Non-word character
\s	Space
\S	Non-space

JSON in Python

JSON (JavaScript Object Notation) is a lightweight format used for storing and exchanging data between applications.

Python provides the built-in json module to work with JSON data.

Why Do We Need JSON?

JSON is widely used because:

- Easy to read
- Easy to write
- Lightweight
- Language independent
- Used in APIs

Importing the Module:

```
import json
```

JSON Functions:

1. json.dumps() :Converts a Python object into a JSON string.

Syntax

```
json.dumps(object)
```

Example:

```
import json
data = {
    "name": "Priya",
    "age": 22
}
print(json.dumps(data))
```

output: {"name": "Priya", "age": 22}

2. json.loads(): Converts a JSON string into a Python object.

Syntax

```
json.loads(json_string)
```

Example:

```
import json
data = '{"name": "Priya","age": 22}'
print(json.loads(data))
```

Output: {'name': 'Priya', 'age': 22}

3. json.dump(): Writes Python data directly into a JSON file.

Syntax

```
json.dump(data, file)
```

Example:

```
import json
data = {
    "name": "Priya",
    "age": 22
}
with open("student.json","w") as file:
    json.dump(data,file)
```

Output (student.json)

```
{  
  "name": "Priya",  
  "age": 22  
}
```

4. json.load(): Reads JSON data from a file and converts it into a Python object.

Syntax

```
json.load(file)
```

Example:

```
import json  
with open("student.json","w") as file:  
    data = json.load(file)  
print(data)
```

Output :{'name': 'Priya', 'age': 22}

You can convert Python objects of the following types, into JSON strings:

- dict
- list
- tuple
- string
- int
- float
- True
- False
- None

Example:

Convert Python objects into JSON strings, and print the values:

```
import json  
print(json.dumps({"name": "John", "age": 30}))  
print(json.dumps(["apple", "bananas"]))  
print(json.dumps(("apple", "bananas")))  
print(json.dumps("hello"))  
print(json.dumps(42))  
print(json.dumps(31.76))  
print(json.dumps(True))  
print(json.dumps(False))  
print(json.dumps(None))
```

output: {"name": "John", "age": 30}

["apple", "bananas"]

["apple", "bananas"]

"hello"

42

31.76

true

false
null

When you convert from Python to JSON, Python objects are converted into the JSON (JavaScript) equivalent:

Python	JSON
dict	Object
list	Array
tuple	Array
str	String
int	Number
float	Number
True	true
False	false
None	null

Example:

Convert a python object containing all the legal data type:

```
import json
x = {
    "name": "John",
    "age": 30,
    "married": True,
    "divorced": False,
    "children": ("Ann","Billy"),
    "pets": None,
    "cars": [
        {"model": "BMW 230", "mpg": 27.5},
        {"model": "Ford Edge", "mpg": 24.1}
    ]
}
print(json.dumps(x))
```

output: {"name": "John", "age": 30, "married": true, "divorced": false, "children": ["Ann", "Billy"], "pets": null, "cars": [{"model": "BMW 230", "mpg": 27.5}, {"model": "Ford Edge", "mpg": 24.1}]}

Format the Result:

The example above prints a JSON string, but it is not very easy to read, with no indentations and line breaks.

The `json.dumps()` method has parameters to make it easier to read the result:

Example :

Use the indent parameter to define the numbers of indents:

```
import json
x = {
    "name": "John",
    "age": 30,
    "married": True,
    "divorced": False,
    "children": ("Ann","Billy"),
    "pets": None,
    "cars": [
        {"model": "BMW 230", "mpg": 27.5},
        {"model": "Ford Edge", "mpg": 24.1}
    ]
}
print(json.dumps(x, indent=4))
```

```
output: "cars": [
    {
        "model": "BMW 230",
        "mpg": 27.5
    },
    {
        "model": "Ford Edge",
        "mpg": 24.1
    }
]
```

You can also define the separators, default value is `(", ", ": ")`, which means using a comma and a space to separate each object, and a colon and a space to separate keys from values:

Example :

Use the separators parameter to change the defaults separator:

```
import json
x = {
    "name": "John",
    "age": 30,
    "married": True,
    "divorced": False,
    "children": ("Ann","Billy"),
    "pets": None,
    "cars": [
        {"model": "BMW 230", "mpg": 27.5},
        {"model": "Ford Edge", "mpg": 24.1}
    ]
}
```



```

    ]
}
print(json.dumps(x, indent=4, separators=(". ", " = ")))

```

output: "cars" = [

```

    {
        "model" = "BMW 230".
        "mpg" = 27.5
    }.
    {
        "model" = "Ford Edge".
        "mpg" = 24.1
    }
]

```

Order the Result:

The json.dumps() method has parameters to order the keys in the result:

Example:

Use the sort_keys parameter to specify if the result should be sorted or not:

```

import json
x = {
    "name": "John",
    "age": 30,
    "married": True,
    "divorced": False,
    "children": ("Ann","Billy"),
    "pets": None,
    "cars": [
        {"model": "BMW 230", "mpg": 27.5},
        {"model": "Ford Edge", "mpg": 24.1}
    ]
}
json.dumps(x, indent=4, sort_keys=True)

```

No output(Empty)

Difference Between dump() and dumps()

Function	Purpose
dump()	Writes JSON to a file
dumps()	Converts Python object to JSON string

Difference Between load() and loads()

Function	Purpose
load()	Reads JSON from a file
loads()	Reads JSON from a string